ENGINEERING GUIDE & VERIFICATION MANUAL

ResQ: Software Testing Architecture

A comprehensive technical breakdown of how testing works in ResQ: the testing pyramid, runtime execution mechanics, architectural invariant verification, and practical runbooks.

125 TESTS • 100% PASS

Vitest + Supertest + Playwright
November 2026

Project: ResQ Crisis Coordination Platform

Focus: Software Mechanics, Assertions, Invariants & Runtime Execution

Coverage: 16 Feature Domains Across 24 Test Files

UNIT TESTS (VITEST)	INTEGRATION (SUPERTEST)	E2E SPECS (PLAYWRIGHT)	AST VERIFIED DISCREPANCIES
17	**101**	**7**	**0**
Algorithms & Math	API, DB & WebSockets	Headless Chromium	Mechanical Parity

1. THE TESTING PYRAMID IN RESQ

Software testing in ResQ follows the classic **Testing Pyramid** design philosophy. The goal is to provide maximum defect detection confidence while keeping test suite execution rapid and deterministic. Rather than relying on slow, brittle manual clicking, ResQ enforces mathematical invariants at the bottom, verifies HTTP/database protocols in the middle, and drives realistic multi-user crisis simulations at the top.



TEST LAYER	CORE TOOLING	WHAT RUNS UNDER THE HOOD	SOFTWARE EXECUTION SPEED
Tier 1: Unit	Vitest + TypeScript (tsx)	Executes pure functions in an isolated Node.js V8 process. Zero I/O, zero network, zero database connections.	SUB-MILLISECOND ~10–50 ms total
Tier 2: Integration	Vitest + Supertest + Mongoose	Mounts Express application on ephemeral OS port; connects to live MongoDB Atlas; drives real HTTP requests and Socket.IO handshakes.	FAST ~24.5s for 101 tests across 21 files
Tier 3: End-to-End	Playwright + Chromium Engine	Launches headless/headed Chromium browsers via Chrome DevTools Protocol (CDP); spawns 4 concurrent browser contexts simultaneously.	THOROUGH ~1–2 min for full core workflows

2. TIER 1: UNIT TESTING (MATHEMATICAL INVARIANTS & MODELS)

A. What It Checks

Unit tests in ResQ verify complex business logic, mathematical algorithms, and deterministic calculations without any external runtime dependencies:

- **4-Factor Matching Engine (`server/src/services/matching.ts`):** Evaluates the Haversine distance formula and combines proximity, suitability, capacity, and availability weights:

  ```
  Score = (0.35 × Proximity) + (0.30 × Suitability) + (0.20 × Capacity) + (0.15 × Availability)
  ```

  Assertions checked: Responders 1 km away score higher than those 25 km away; responders at full capacity receive a capacity score penalty; unavailable responders are down-ranked.
- **Location Privacy & Obfuscation (`server/src/utils/privacy.ts`):** Verifies that Gaussian perturbation adds approximately 500m of random jitter to exact coordinates:

  ```
  const distanceDelta = calculateHaversine(preciseCoords, publicFuzzedCoords);
  expect(distanceDelta).toBeGreaterThanOrEqual(0.15); // At least 150m away
  expect(distanceDelta).toBeLessThanOrEqual(0.85); // At most 850m away
  ```

  Assertions checked: The fuzzed location is never identical to the precise coordinate; the delta remains tightly bounded within safe spatial margins.
- **AI Triage Classification Keywords (`server/src/services/classification.ts`):** Checks that distress text such as *"Elderly patient needs cardiac meds"* produces category `medical` with high confidence, while ambiguous text falls back cleanly without crashing.

B. How It Runs Under the Hood

```
cd server npx vitest run tests/matching.test.ts
```

Vitest compiles the TypeScript files into memory using `esbuild`, invokes the functions directly with sample fixtures, executes assertions using Chai-compatible `expect()` primitives, and prints execution times per test case.

3. TIER 2: INTEGRATION TESTING (SUPERTEST + EXPRESS + MONGODB)

A. What It Checks

Integration tests exercise the full backend stack: routing, authentication middleware, controller logic, database schema validations, and Socket.IO event emissions.

- **Role-Based Access Control (RBAC) Bi-Directional Enforcement:** Every protected route is tested in *both* directions:

ENDPOINT	ROLE UNDER TEST	EXPECTED HTTP CODE	VERIFICATION ASSERTION
PATCH /api/requests/:id/claim	volunteer	HTTP 200 OK	Request status transitions to in_progress; RequestAssignment created.
PATCH /api/requests/:id/claim	affected_person	HTTP 403 FORBIDDEN	Middleware blocks caller before touching database; error message verified.
PATCH /api/requests/:id/verify	admin or verified ngo	HTTP 200 OK	Creates formal Verification document with inspection remarks & priority.
PATCH /api/requests/:id/verify	volunteer	HTTP 403 FORBIDDEN	Field volunteers cannot self-verify distress tickets.

- **Enforced State Machine Lifecycle:** Validates legal state progressions (unverified $ ightarrow$ verified $ ightarrow$ in_progress $ ightarrow$ resolved $ ightarrow$ closed). Asserts that illegal transitions (e.g., trying to jump directly from unverified to closed) are rejected with HTTP 400 Bad Request.
- **Inventory Reservation & Concurrency:** When a provider reserves 5 blankets, asserts that quantityReserved increments by 5 while quantityAvailable decrements by 5. When a ticket is cancelled, asserts that the reserved balance is automatically restored to available stock.
- **Real-Time Socket Event Privacy Masking:** Spawns two real Socket.IO clients: Client A (anonymous) and Client B (authenticated volunteer). Triggers a claim and asserts that Client A receives contactPhone: "REDACTED" and fuzzed GPS, while Client B receives unmasked details.

B. How It Runs Under the Hood

```
[Vitest Test Runner] | ├── 1. beforeAll: Connects to MongoDB Atlas cluster via Mongoose ├── 2. Seeds isolated documents with random tracking IDs (e.g. REQ-2026-XXXXXX) ├── 3. Supertest wraps the Express 'app' instance in an ephemeral HTTP server ├── 4. Executes supertest(app).patch('/api/requests/...').set('Authorization', token) ├── 5. Queries MongoDB collection directly to prove database state was mutated └── 6. afterAll: Cleans up created test documents and closes database connection
```

4. TIER 3: END-TO-END TESTING (PLAYWRIGHT BROWSER AUTOMATION)

A. What It Checks

Playwright executes complete user journeys in real Chromium browser instances. Its unique role in ResQ is testing **multi-user concurrent coordination** — a situation impossible to test reliably with unit tests alone.

> **Multi-Context Concurrency:** Playwright spawns 4 isolated browser contexts (independent cookie jars, localStorage, and WebSocket sessions) simulating 4 distinct people operating on their devices simultaneously.

```
[Playwright Chromium Engine] | ├── Context A (Citizen Browser): | • Navigates to /submit | • Fills emergency description, selects category, and submits ticket | • Waits on tracking page /track/REQ-2026-XXXXXX | ├── Context B (Dispatcher / Admin Browser): | • Logs in as admin@resq.org | • Watches live dispatch rail update via Socket.IO WITHOUT page refresh | • Clicks "Verify" with priority "High" | ├── Context C (Field Volunteer Browser): | • Logs in as rahul.volunteer@example.org | • Watches verified ticket appear in claimable list live | • Clicks "Claim this request" | └── Concurrent Assertions Across Contexts: • Asserts Citizen's screen (Context A) automatically turns to "IN_PROGRESS" • Volunteer sends message in comms thread: "En route with medical supplies" • Asserts Citizen receives message live in thread without browser reload • Asserts Context D (Unrelated Supplier) receives 0 messages (comms isolation) • Volunteer clicks "Mark resolved" ➔ Citizen clicks "Confirm safe delivery" • Asserts ticket transitions to terminal "CLOSED" status
```

B. How It Runs Under the Hood

Playwright communicates directly with Chromium via the **Chrome DevTools Protocol (CDP)**. It waits for DOM stability (auto-waiting for network idle, visibility, and element interactability), eliminating fragile timing bugs:

```
await pageAdmin.click('button:has-text("Verify")'); await expect(pageCitizen.locator('.status-chip')).toHaveText('VERIFIED');
```

5. MECHANICAL TEST INVENTORY & STATIC TRACEABILITY VALIDATION

To prevent documentation drift, ResQ includes an automated verification script (`scripts/generate-test-inventory.ts`). This script acts as a static analysis tool:

- **AST Test Discovery:** Uses the TypeScript Compiler API to parse test files and discover every `it()` and `test()` block, recording exact file paths, line numbers, and full enclosing describe titles.
- **Cross-Reference Validation:** Compares discovered tests against `docs/test-case-annotations.json`.
- **Code-Under-Test Verification:** For every entry in `code_under_test`, the script verifies:
  1. The target source file actually exists on disk.
  2. The specified line range (e.g. `454-546`) is within the file's line count.
  3. The named function or route (e.g. `PATCH /api/requests/:id/claim`) actually appears within that line slice.

  If any line range drifts or a function moves, **the script fails with a non-zero exit code**.

6. PRACTICAL TESTING RUNBOOK & COMMANDS

1. Running All Backend Tests (118 Tests)

```
cd "code/server" npm test
```

Executes Vitest across all 21 test files. Reports pass/fail, duration, and test suite counts.

2. Running a Specific Test File (Targeted Debugging)

```
cd "code/server" npm test tests/cancellation.test.ts npm test tests/socket_broadcast.test.ts npm test tests/allocation.test.ts
```

3. Running Playwright End-to-End Tests (7 Scenarios)

Prerequisite: Ensure dev servers are running (Backend on `:5000`, Frontend on `:5173`).

```
cd "code/client" npx playwright test
```

4. Running Playwright in Headed Mode (Watch Real Browsers Open)

Opens real visible Chromium windows so you can watch automated clicks and multi-context sync live:

```
cd "code/client" npx playwright test --headed
```

5. Running Playwright with Interactive UI Mode

Provides time-travel debugging, step-by-step DOM snapshots, console logs, and network inspection:

```
cd "code/client" npx playwright test --ui
```

6. Regenerating & Validating Test Inventory

```
cd "code" npm run inventory
```

Scans the repository, runs AST validations, verifies all 125 test annotations against source code lines, and updates `docs/TEST_CASES.md` and `docs/test-inventory.json`.

7. SUMMARY: WHAT GIVES CONFIDENCE IN RESQ?

ASPECT	HOW IT IS PROVEN	AUTOMATED METRIC
Security & RBAC	Tested bi-directionally: unauthorized roles receive HTTP 403, authorized roles succeed.	15 Auth & Role tests
Life-Safety Privacy	Gaussian coordinate offset verified mathematically; telephone numbers masked on public endpoints.	11 Privacy tests
State Machine Integrity	Every status transition enforced; illegal jumps rejected; requester confirmation required for closure.	12 Lifecycle tests
Concurrency Safety	Atomic query filters (`{ status: 'verified' }`) prevent split-brain double-claims.	0% Double-booking rate
Real-Time Reliability	Multi-context Playwright tests prove Socket.IO updates DOM across different users with zero refresh.	7 Playwright E2E specs